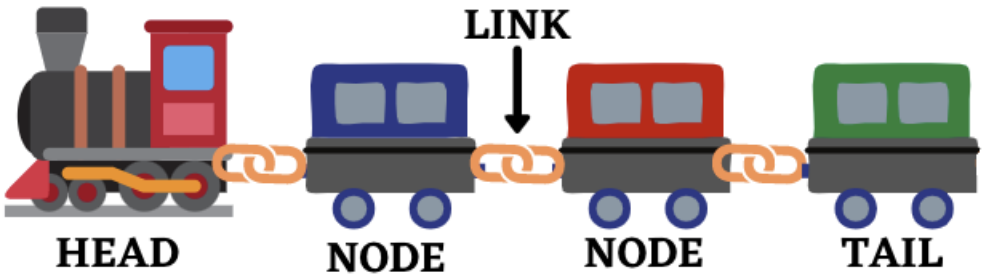
THE IRON NEXUS

Real-Time Railway Management System

Data Structures | C++ Implementation



Project Overview

The Railway Authority is facing a crisis. Their legacy systems, built on fragmented spreadsheets and manual logs, can no longer handle the complexity of a modern, high-speed rail network. As the Lead Software Architect, you have been commissioned to build "**The Iron Nexus**" a comprehensive, real-time system.

Your mission is to design and develop a digital system that models the complete lifecycle of a railway environment from the registration of trains, to the organization of their coaches, the mapping of the railway network, and the management of individual seat bookings. This system should not only simulate real-world operations but also serve as a platform for understanding how different data organization strategies behave under dynamic conditions. Through this project, you are expected to design and implement appropriate data structures for solving a real-world train management problem. As you build and refine your system, you will also develop the ability to analyze program behavior and identify bugs related to data structure operations, especially when handling dynamic updates, edge cases, and complex interactions between different components. In essence, this project is not just about building a working

application, it is about making informed design decisions, evaluating their impact, and continuously improving your system by identifying and correcting flaws in your implementation.

In the world of high stakes infrastructure, efficiency isn't just a technical metric, it is the difference between a thriving economy and a total gridlock. When we talk about "logic skeletons," we are talking about the invisible architecture that prevents the entire system from collapsing under its own weight. Efficiency is the priority. If a train lookup is too slow, the system fails. If a route isn't optimized, fuel is wasted. If a seat booking is lost, the public loses trust. Your goal is to choose and implement the most efficient logic skeletons to ensure the system runs without a hitch.

System Architecture

Module 1: Train Registry

Imagine an operator at the Railway Authority. They need to locate a specific train among 10,000 active trains to issue an emergency stop command. Each train has a unique TrainID, a name, and its own configuration of cars. If you store these trains in a simple list, the computer must check every single train one by one. In computer science terms, this is linear time. If the train is at the end of the list, the lookup takes too long, the command is late. The Challenge: You need a structure that organizes trains by their ID. As the fleet grows or as old trains are decommissioned, the depth of your search must remain consistent. Think of a self-balancing organizational hierarchy. You must be able to insert new trains, remove retired ones, and perform various "patrols" (traversals) of the entire fleet.

Module 2: Coach Management

A train is not a static object; it is an assembly of individual coaches. Railway staff frequently modify the composition based on demand. A new coach might be attached at the front for

VIP passengers, an extra economy coach might be inserted somewhere in the middle, or a coach can be added at end. Occasionally, specific coaches are removed for maintenance. An engineer must be able to walk from the engine to the caboose and back again. Sometimes, the entire orientation of the train must be reversed for a return journey. Here it needs to preserves order, supports flexible modifications, and allows smooth movement across the train in either direction.

Module 3: Railway Track Network

The railway doesn't just go in a straight line. It is a web of stations connected by tracks of varying lengths and qualities. Every mile a train travels on a non-optimized route represents thousands of gallons of wasted fuel and unnecessary wear on the tracks. You must map the entire country (limit your simulation to 15 major hubs). The system must calculate the most efficient path between two distant cities based on track distance or travel time. Railway planners often modify the network as well. New stations and routes are introduced, stations and routes are closed, and alternative connections are established. The system must adapt dynamically to these changes while still being able to provide accurate and optimal path suggestions.

Module 4: Seating Chart

Public trust is the hardest thing to build and the easiest to break. A traveler who pays for a premium seat only to find someone else sitting there because of a system glitch will never use the railway again. Inside every coach lies the most complex part of the system: seat management. Seats are numbered continuously across the whole train. Passengers frequently book and cancel seats, and the system must respond instantly. A passenger might inquire about a specific seat’s availability, expecting an immediate response. Meanwhile, railway staff may need to view all booked or available seats in an organized

manner, often sorted for clarity. On one hand, the system must support extremely fast lookups to check or update a seat's status. On the other hand, it must maintain a structured view of seats that allows ordered traversal and display. You must implement a dual-layer system.

Layer 1

You can take the seat number as key and run it through a specific formula. This formula spits out an exact index in memory. Because the computer goes straight to that coordinate, the time it takes to find a seat is constant. Occasionally, the math might accidentally give the same index for two different seats. Your job as the architect is to handle this.

Layer 2

Implement structure starts with a single node. Every time you add a seat, the system makes a choice: Is this seat number smaller than the current one: Go to the left. Is it larger: Go to the right.

Module 5: Operation Logging System

Behind the scenes, every action performed in the system is recorded for accountability and traceability. Whether a train is created, a coach is added, or a seat is booked, these events are stored in a log that reflects the system's history. This is not just a simple record-keeping task. The sequence of events must remain logically consistent. For instance, it would make little sense for a log to show a coach being added to a train that no longer exists. Therefore, the system must ensure that recorded actions follow a meaningful and valid progression. Operators also wish to review recent actions first, especially when diagnosing issues or verifying operations. It is important to take care of these points and implement specific structure that full fill these requirements.

Storage & Interaction

The Railway Authority uses a command-line interface. There must be an option to save the data of the entire system at once, as well as for each module individually, such as the train registry, coach details, and seat booking records, to a file (e.g., in the form of matrices, arrays, or structured formats). The file storage must be consistent, and there must be an option to load the data back into the system from the file.

Engineering Constraints

To prove your worth as a Lead Architect, you must build this from the ground up. You are strictly prohibited from using pre-built containers like <vector>, <map>, <list>, or <stack>.

You must manage your own memory using pointers and custom classes. You must handle edge cases. What happens if a user tries to book a seat that doesn't exist? What if two stations aren't connected? Your system must be crash-proof.

Key Objectives

- Design and implement appropriate data structures for solving a real-world train management problem.
- Strengthen understanding of Trees, Graphs, Hashing, and Linked Lists.
- Develop problem-solving skills without relying on STL.
- Analyze program behavior and identify bugs related to data structure operations.

Submission Requirements

- Complete source code
- Executable file
- One-page Report: states which data structure(s) is used for each module and the justification.

Bonus

- Support for single-step undo and redo operations
- Innovation with Functional Efficiency, novel optimization strategies
- While saving data to file create an additional file that save data to a file in a format that preserves its original structure, such as storing a Binary Search Tree (BST) in a way that retains its hierarchical tree form.
- Colored and structured I/O on terminal.

Academic Integrity

Each group must implement their own independent solution.

Any violation of academic integrity will result in **ZERO marks** for all involved parties.